Module-I
Lecture-III
Transformations for High Level Synthesis

1. Introduction

In Lecture 1 of Module 1, we discussed that for any VLSI design we start with specifications and the first step is to obtain the Register Transfer Level (RTL) circuit. RTL circuit is obtained from specifications using High Level Synthesis (HLS) algorithms. As specifications are processed by HLS algorithms, they need to be represented using some modeling language. In Lecture 2, we discussed Control and Data Flow Graph (CDFG), which is one of the most widely accepted modeling paradigm for specifications that are processed by HLS tools.

In Lecture 2, we saw several examples where specifications written in terms of Verilog were translated into CDFGs. In the examples, there was almost one-to-one mapping of the lines of Verlog codes with CDFG nodes and edges. It must be noted that specifications and HDL codes are written by humans, which have errors, redundancies and may be inefficient. So, before processing the CDGFs using HLS tools we need to do some transformations to eliminate redundancies, inefficiencies etc. In this lecture, we will discuss some well-known transformations made in the CDFGs, which make them more amiable for HLS.

The transformations that can be performed on CDFGs can be classified into the following broad categories:

1. Compiler based transformations [1]
2. Flow-graph based transformations [2]
3. Hardware library based transformations [3]

Now, we will discuss these transformations, presenting examples and providing the corresponding transformation procedures.

2. Compiler based transformations

In programming paradigm, "code optimization" is a very important step carried out by the compiler. Apart from syntax checking, semantic checking and code generation, the compiler also improves the quality of a program in terms of runtime; this is called code optimization. Program improvement may comprise change of instruction sequence, elimination of instructions, changes in instructions itself, while retaining the meaning of the original code; these changes are called code transformations. Code optimization process has four components: (i) discovering opportunities to apply a transformation, (ii) proving that the transformation can be applied safely at those sites, (iii) ensuring that its application is profitable, and (iv) actually rewriting the code.

The same philosophy can also be applied to transform a CDFG that models hardware. In case of software, code optimization reduces runtime by eliminating/reducing unnecessary computations and in case of hardware, code optimization would reduce circuit modules thereby resulting in lower area, lower power and high frequency (i.e., less computation). In this section, we will discuss compiler based transformations for CDFGs.

2.1 Loop Invariant computations and code hoisting

An expression evaluated inside a loop that uses operands whose values do not change from iteration to iteration is called a loop invariant computation. So the evaluation of a loop invariant expression can be moved out of the loop. If the loop's body executes more than once, this should reduce the number of times the expression is evaluated. Further, the scheme also speeds up overall system clock because time required to evaluate a loop decreases (as hardware for computing the invariant code is removed out of the loop).

Figure 1 illustrates a Verilog code having a loop invariant computation. It may be noted that the computation "F = E + B;" is loop invariant, because the operands (E and B) used is the computation (to calculate F) do no change with iteration of the while loop. So we can bring the loop invariant computation "F = E + B;" before the loop begins; this is called code hosting. Figure 2 illustrates CDFG for

the Verilog code, first with the loop invariant computation in the loop and then with code hoisting.

```
module CDFG_tr_example (A,B,C,D,E);
input [3:0] B,C,D,E;
 reg [7:0] A,F;
output [7:0] A,F;
        initial begin A = B * C + D; end
        while ( A > 0 ) begin
        A  = A − 1;
        F = E + B;
       end
endmodule
```

Figure 1. Verilog code with loop invariant computation

The conditions required to determine loop invariant computations are as follows.
Any computation (or expression) can be written as
"assigned variable= operand1 OPERATOR operand2… OPERATOR operandn"
In terms of a CDFG, in the computation, "assigned variable" and "operand1" through "operandn" correspond to storage nodes while "OPERATOR"s corresponds to operational nodes.
A computation "assigned variable=(operand1,operand2,……,operandn)" that is inside a loop, is loop invariant if

(1) If all the operands are constants
(2) All the computations that assign values to the operands are located outside the loop
(3) All the computations that assign values to the operands are themselves loop invariant

In the above example (Figure 1), “F = E + B;” is loop invariant because computations that assign values to the operands (F and B) are located outside the loop (taken from inputs).

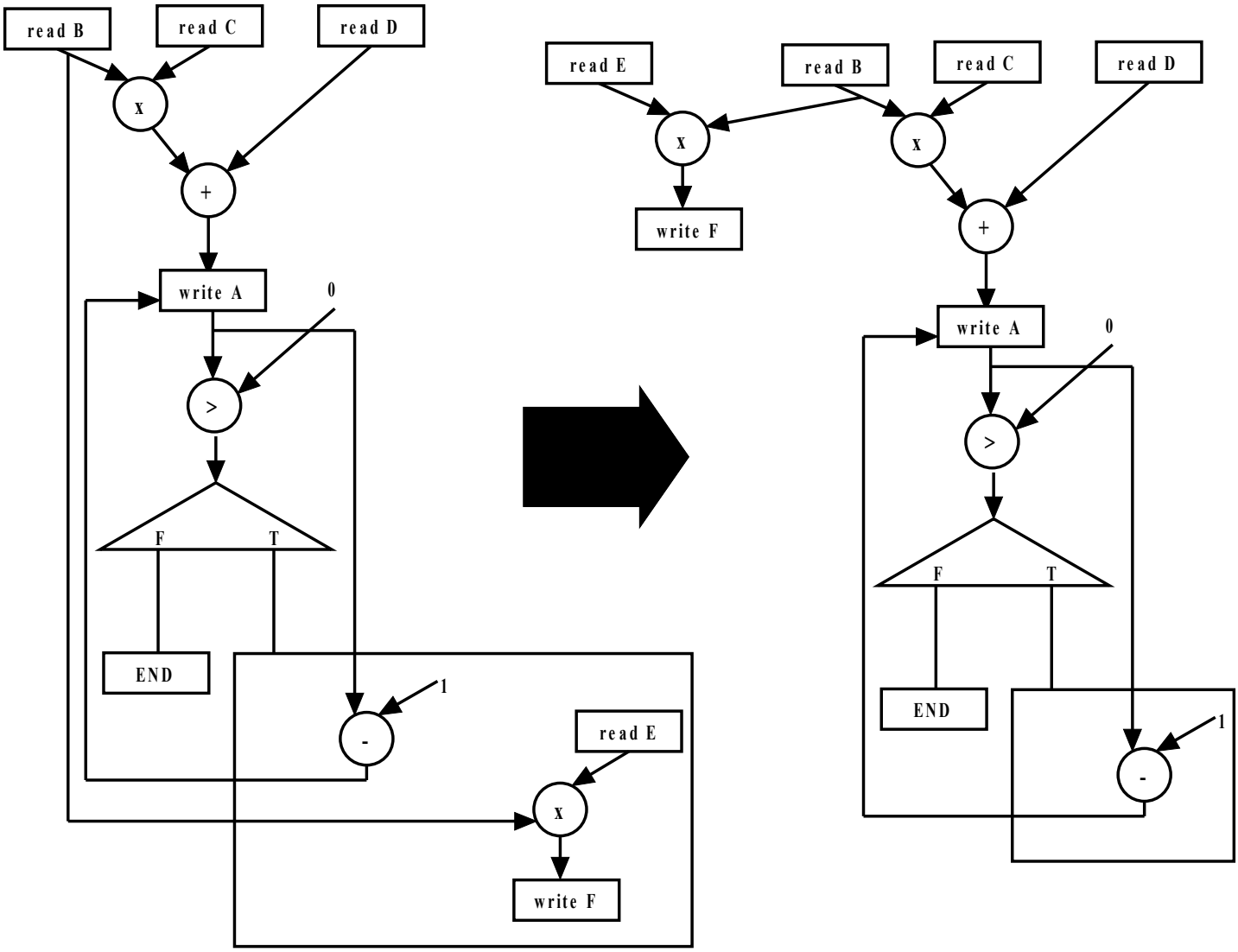


Figure 2. CDFG transformation: Loop Invariant computation and Code hoisting

2.2 **Constant folding**

An expression

“assigned variable= operand1 OPERATOR operand2, … OPERATOR operandn” where all the operands are constants can be replaced with the pre-computed final result directly written to assigned variable. This principal of code optimization is called constant folding. Expressions, which involve computations over constants, always generate the same output irrespective of inputs. So the output of the computation over constants can be pre-computed by looking statically at the code and that output value can be directly used. So resources required for computing such expressions can be avoided.

Figure 3 illustrates a Verilog code having an expression where all the operands are constants. It may be noted that the computation “A = 1 + 2;” always results in A having the value of 3, irrespective of inputs. Also the final value of A (i.e., 3) can be pre-computed by statically looking at the code. Finally, the expression “A = 1 + 2;” can be replaced with “A = 3;”, which saves an adder and a register.

Figure 4 illustrates CDFG transformation using constant folding for the Verilog code (Figure 3).

```
module CDFG_trexample (A);
reg [3:0] A;
output [3:0] A;
        initial begin A = 1 + 2 ; end
       end
endmodule
```

Figure 3. Verilog code with a computation where all operands are constants

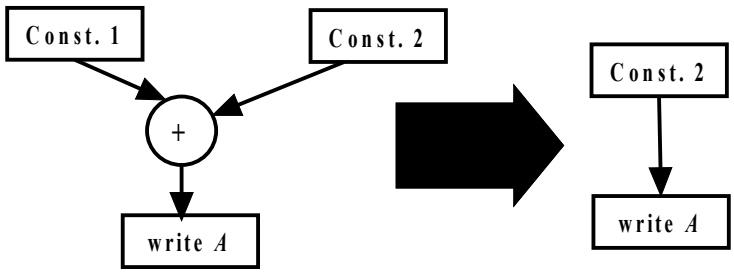


Figure 4. CDFG transformation: Constant folding

2.3 **Redundant computation elimination**

If there are two expressions

"*assigned variable1=operand11 OPERATOR11,..,OPERATOR1(n-1) operand1n*"

and

"*assigned variable2=operand21 OPERATOR21,..,OPERATOR2(n-1) operand2n*"

where,

operand11=operand21, OPERATOR12=OPERATOR22, ..., operand1n= operand2n

and the operands of the computations do not get modified in between evaluating "assigned variable1" and "assigned variable2", then we can perform the computation (i.e., "operand11 OPERATOR11 ……OPERATOR1n operand1n") only once and assign "assigned variable2"= "assigned variable1". This reduction is called redundant computation elimination. Obviously, redundant computation elimination reduces hardware requirements.

Figure 5 illustrates a Verilog code having an expression with redundant computation. It may be noted that computations "C = A + B" and "D = A + B" will result in same value assigned to C and D. This is because operands and operators of the both the computations are same and operands A, B are not changed in between "C = A + B" and "D = A + B". Figure 6 illustrates CDFG transformation using redundant computation elimination for the Verilog code (Figure 5).

```
module CDFG_tr_example (A,B,C,D);
input [3:0] A,B;
 reg [3:0] C,D;
output [3:0] C,D;
          initial begin
          C = A + B;
          D = A + B;
         end
endmodule
```

Figure 5. Verilog code with redundant computation

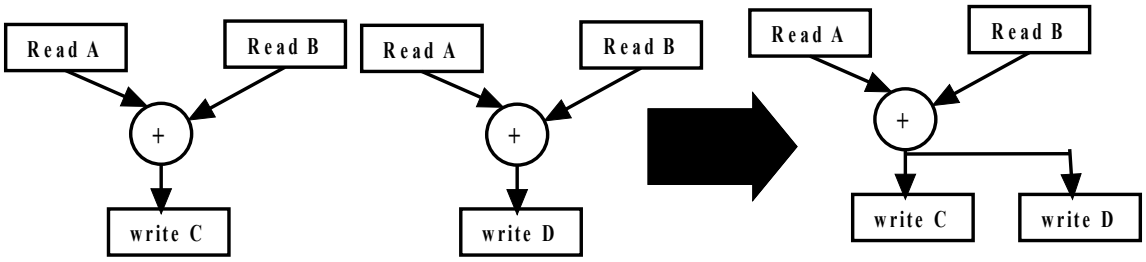


Figure 6. CDFG transformation: Redundant computation elimination

2.4 **Common Sub-computation elimination**

If there are two expressions

"*assigned variable1=operand11 OPERATOR11 ……OPERATOR1n operand1n*"

and

"*assigned variable2=operand21 OPERATOR21 ….,OPERATOR2n operand2n*"

where,

operand1k=operand2k, OPERATOR1k=OPERATOR2k,…., operand1m= operand2m

and $1 \le k, m \le n$,

and the operands *operand1k, operand1(k+1),….. operand2m* do not get modified in between evaluating "assigned variable1" and "assigned variable2", then we can perform the sub-computation (i.e., "*operand1k OPERATOR1k,…., OPERATOR1(k-1) operand2m*") only once for "assigned variable1" and use the value for "assigned variable2". This reduction is called common sub-computation elimination. So, redundant computation elimination is a special case of common sub-computation elimination, where $k = 1,$ and $m = n$. Like redundant computation elimination, common sub-computation elimination also reduces hardware requirements.

There is a small change in common sub-computation elimination procedure compared to redundant computation elimination. In common sub-computation elimination procedure we need to have an extra temporary variable which stores the value of the common sub-computation for future use.

Figure 7 illustrates a Verilog code having a common sub-computation. It may be noted that computations "D = A + B + C" and "F = A + B + E" have "A+B" as the common sub-computation. This is because operands and operators of the both the sub-computations are same and operands A, B are not changed in between "D = A + B + C" and "F = A + B + E". Figure 8 illustrates CDFG transformation using redundant sub-computation elimination for the Verilog code (Figure 7). It may be noted that in the CDFG a new temporary variable T1 (represented by a storage node) is used to store the value of the common sub-computation and it is used for computing both "D = A + B + C" and "F = A + B + E".



```
module CDFG_tr_example (A,B,C,D,E,F);
input [3:0] A,B,C,E;
 reg [3:0] D,F;
output [3:0] D,F;
          initial begin
          D = A + B + C;
          F = A + B + E;
         end
endmodule
```

Figure 7. Verilog code with common sub-computations

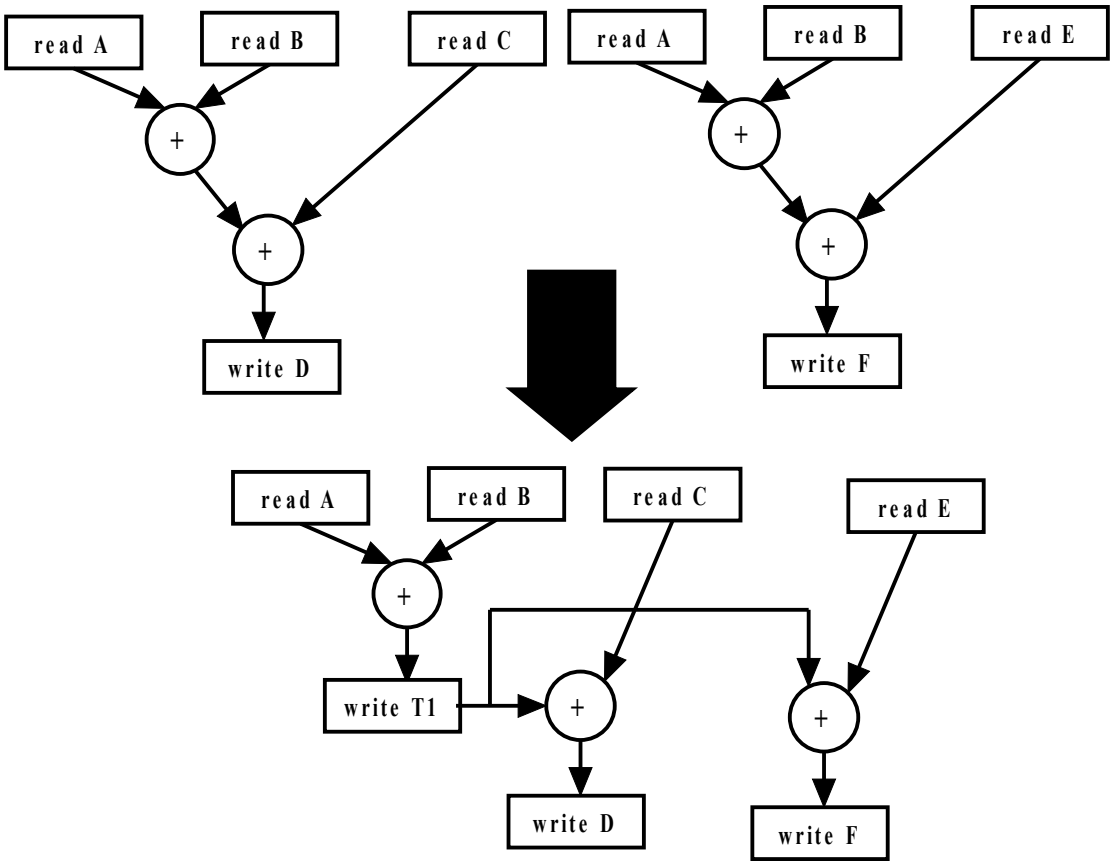


Figure 8. CDFG transformation: Common Sub-computation elimination

2.5 **Dead-computation elimination**

Dead-computation elimination is one of the most simple transformation where a computation that has no effect on the output of a code is eliminated. In the CDFG the corresponding nodes are eliminated. Figure 9 illustrates a Verilog code having a dead-computation. It may be noted that computation "E = A + 1;" is dead because it has no effect on the output of the code. Figure 10 illustrates CDFG transformation using dead-computation elimination (where the nodes in the dotted box are eliminated) for the Verilog code of Figure 9.

```
module CDFG_tr_example (A,B,C,D);
input [3:0] B,C,D;
 reg [3:0] A,E;
output [3:0] A;
        initial begin
        A = B + C + D;
        E = A + 1;
       end
endmodule
```

Figure 9. Verilog code with dead-computations

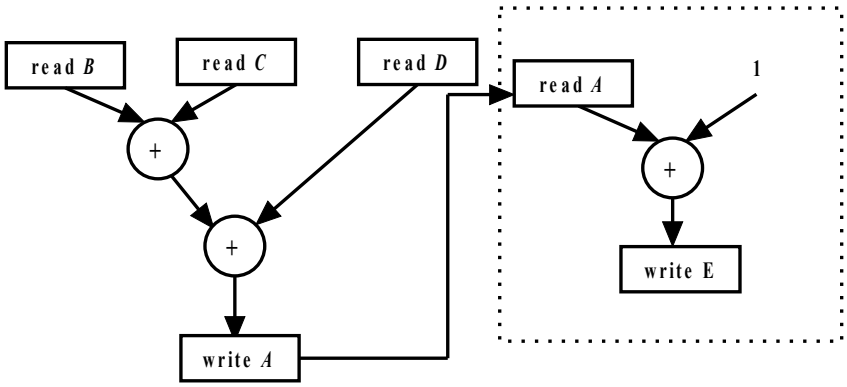


Figure 10. CDFG transformation: Dead computation elimination

3. **Flow-graph based transformations**

In the last section, the transformations we discussed were based on compiler directives and so they could be done using analysis of the HDL codes. In this section, we will discuss transformations that are done on the flow graph (CDFG in this lecture) structure. In flow-graph based transformations the major objecting is to change the graph so that parallelism of the design can be explored. In this section, we will see two types of flow-graph transformations

i. Tree height reduction based transformation

ii. Control flow to data flow based transformation

3.1 **Tree height reduction based transformation**

Tree height reduction based transformation uses the commutative and the distributive properties of expressions to decrease the length. Let there be an expression

"*assigned variable1=operand11 OPERATOR11 ……OPERATOR1n operand1n*", which can be reduced in length by breaking it into sub-computations and storing their outputs in temporary variables. Finally, operations are done on the temporary variables to determine the value of "assigned variable1". It may be noted that evaluation of a long expression involves more steps because operations are done in sequence (on order of precedence of the operators). On the other hand, if we break the expression into sub-expressions then they can be evaluated in parallel, there by speeding up the computation. However, in the case of sequential evaluation, less hardware is required as the same circuit can be reused (in different steps), which is not possible in case of parallel evaluation.

Figure 11 illustrates a Verilog code having a computation "G = A+ B + C + D + E + F ;". This computation requires 1 adder which can be used in a sequential fashion, first adding A with B, then C with the result, and so on. Figure 12 (top) illustrates CDFG for the Verilog code. From the CDFG it may be noted that sequential computation requires 5 steps. Now, if we have more adders, then some of the sub-computations can be done in parallel; A+B, C+D and E+F can be done concurrently and results stored in three temporary variables. Finally G can be determined by adding the temporary variables. This parallel computation takes 3

steps. Figure 12 (bottom) illustrates CDFG after inducing parallelism. It might be noted that induction of parallelism reduces tree height.

```
module CDFG_tr_example (A,B,C,D,E,F,G);
input [3:0] A,B,C,D,E,F;
 reg [3:0] G;
output [3:0] G;
          initial begin
          G = A+ B + C + D + E + F;
         end
endmodule
```

Figure 11. Verilog code with long expression

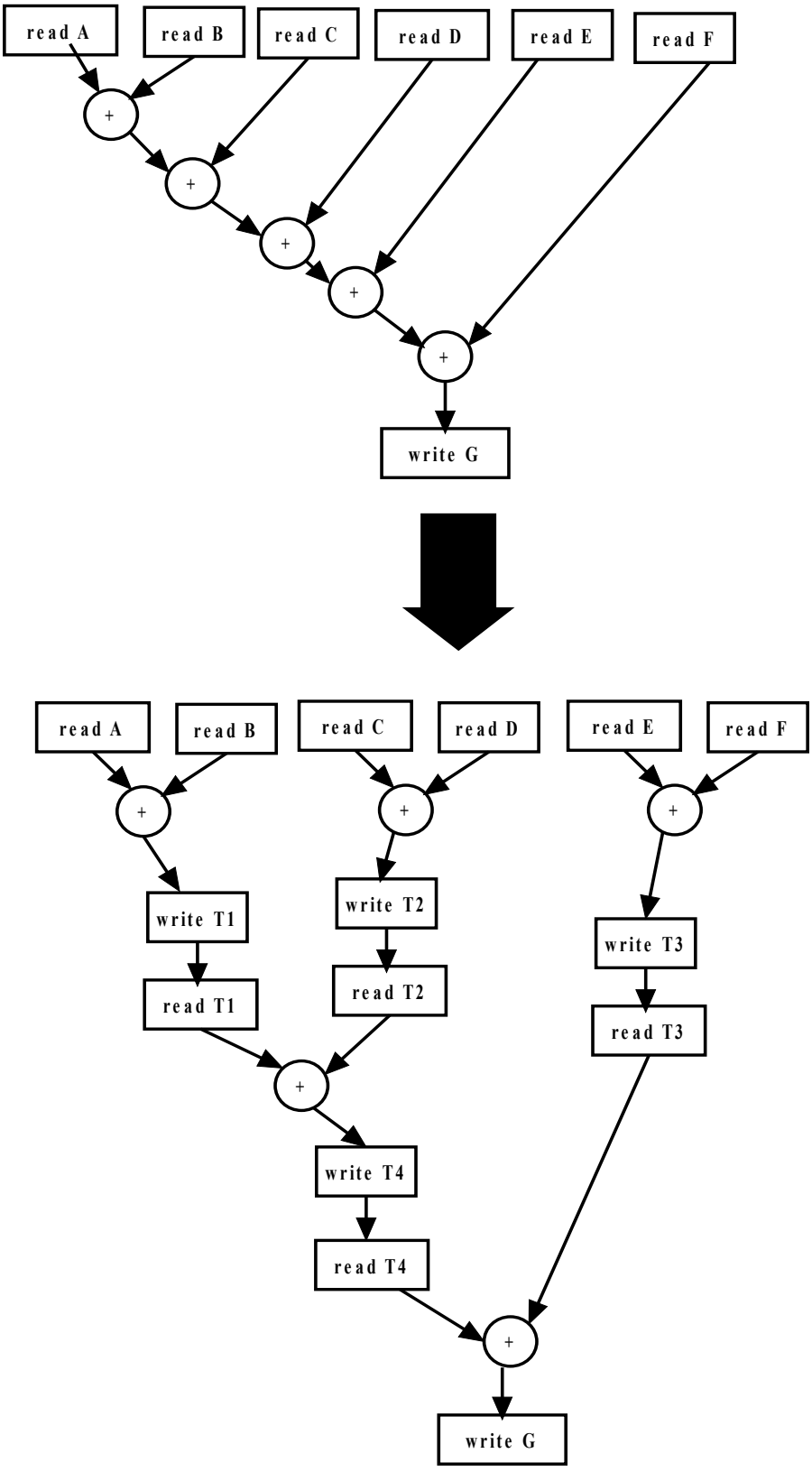


Figure 12. CDFG transformation: Height Reduction for parallelism

3.1 **Control flow to data flow based transformation**

This transformation has already been discussed in Section 4 of Lecture 2 of this module. It was illustrated that dataflow based CDFG is a better representation of hardware.

4. **Hardware library based transformations**

Any operational node of a CDFG has a corresponding circuit capable of performing the operation e.g., adder to do addition, comparator for equality checking etc. This mapping of an operational node with hardware circuit depends on circuits available for implementation in the design library. For example, checking equality of a variable A (6 bit number say) with zero can be done using a 6-bit comparator or simply feeding all the 6 bits of A to an OR gate (if A is 0 then all the bits are 0 thereby generating 0 from the OR gate). The OR gate implementation consumes less hardware than the comparator, however, depends of availability of a 6 input OR gate in the library. Hardware library based transformations are modifications in the operational nodes of a CDFG, depending on availability of corresponding circuits in the design library, for achieving efficient circuit implementation in terms of area, frequency, power etc.

Figure 13 illustrates CDFG for a case structure where one computation is "A=B+1", which involves an operational node for addition; this can be implemented using an adder where one operand is B and the other is 1. This computation can also be implemented using "INCR" circuit [3], which increments the input by 1. INCR block involves less resources than an adder. So, if the library has INCR block then CDFG can be transformed by replacing the adder operational node with INCR operational node (Figure 13, right side).

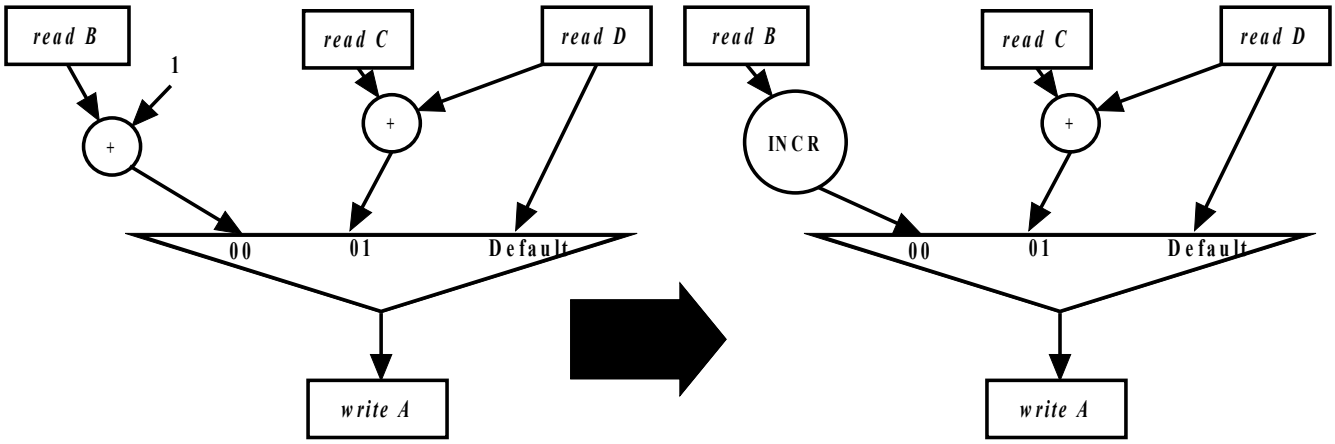


Figure 13. Hardware Library Specific Transformation using INCR

Figure 14 illustrates CDFG for computation “B=A*2”, which involves an operational node for multiplication; this can be implemented using a multiplier where one operand is A and the other is 2. This computation can also be implemented using “left shift” circuit [3], which multiplies the input by 2. “Left shift” block is a shift register and therefore involves much less resources than a multiplier. So, if the library has “left shift” block then CDFG can be transformed by replacing the multiply operational node with “left shift” operational node (Figure 14, right side).

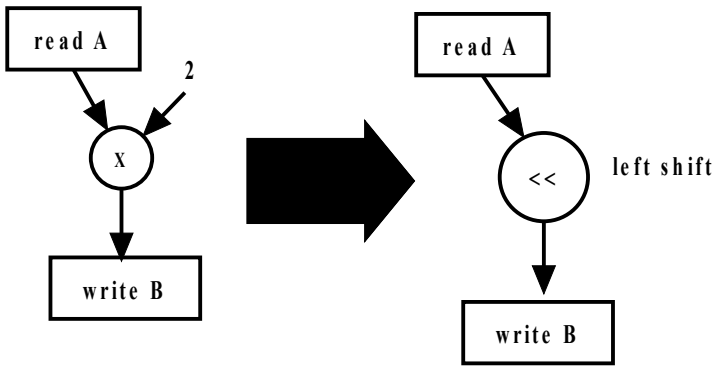


Figure 14. Hardware Library Specific Transformation using shifting

5. **Conclusions**

In this lecture, we have discussed several transformations on HDL codes and CDFGs, which aims at efficient hardware implementation in terms of area, frequency, power etc. Once the transformed CDFG is obtained it is processed by HLS algorithms comprising three steps namely, scheduling, allocation and binding. The next module focuses on these steps of HLS.

Question and Answers

1. After transformation of CDFGs and HDL codes, what step is required before processing them through HLS tools?

Answer:

After the CDFG / HDL code is transformed, it needs to be verified if their meaning (i.e., input-output behavior) remains same. In other words formal verification schemes are required to establish equivalence between the original and transformed CDFG/ HDL code. Although the techniques discussed above preserve equivalence, however, sometimes some parts of specifications may be missed resulting in behavioral difference. For example, sometimes dead-computations are kept in a loop to increase delay (for synchronization). If dead-code elimination is applied without looking into the timing specification then behavioral difference in terms of timing mismatch may occur.

References

[1]. A. V. Aho, R. Sethi, and J. D. Ullman, “Compilers: Principles, Techniques, and Tools”, Addison-Wesley, 1986.

[2]. D. Gajski, N. Dutt, S. Lin, and A. Wu, “High Level Synthesis: Introduction to Chip and System Design”, Kluwer, 1992.

[3]. Joseph J. F. Cavanagh, “Digital Computer Arithmetic: Design and Implementation”, 1st Edition, McGraw-Hill, Inc, 1983